

# Django REST Framework - JWT Authentication Guide

JSON Web Token (JWT) is a compact, URL-safe way of representing claims between two parties. In Django REST Framework, JWT is commonly used for authentication, replacing traditional session-based authentication. This guide explains how to implement JWT in Django using SimpleJWT, including registration and email verification.

## 1. Installing Dependencies

Install Django REST Framework and SimpleJWT:

```
pip install djangorestframework djangorestframework-simplejwt
```

Add `rest_framework` to `INSTALLED_APPS` in `settings.py`:

```
INSTALLED_APPS = [  
    ...  
    'rest_framework',  
]
```

Configure REST Framework authentication in `settings.py`:

```
REST_FRAMEWORK = {  
    'DEFAULT_AUTHENTICATION_CLASSES': (  
        'rest_framework_simplejwt.authentication.JWTAuthentication',  
    )  
}
```

## 2. Setting up JWT URLs

In `urls.py`, add endpoints for JWT token creation and refresh:

```
from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView  
  
urlpatterns = [  
    path('api/token/', TokenObtainPairView.as_view(), name='token_obtain_pair'),  
    path('api/token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),  
]
```

## 3. User Registration API

You can create a registration view to allow users to sign up.

`serializers.py`:

```
-----  
from django.contrib.auth.models import User  
from rest_framework import serializers  
  
class RegisterSerializer(serializers.ModelSerializer):  
    password = serializers.CharField(write_only=True)  
  
    class Meta:  
        model = User  
        fields = ('username', 'email', 'password')  
  
    def create(self, validated_data):
```

```

user = User.objects.create_user(
    username=validated_data['username'],
    email=validated_data['email'],
    password=validated_data['password']
)
return user

```

views.py:

```

-----
from rest_framework import generics
from .serializers import RegisterSerializer

class RegisterView(generics.CreateAPIView):
    serializer_class = RegisterSerializer

```

#### 4. Email Verification (Optional but Recommended)

To add verification, generate a token and send a verification link via email.

You can use Django's built-in signing module or packages like `django-rest-framework-simplejwt` + custom claims.

Example flow:

- User registers
- System sends email with link containing a signed token
- User clicks link -> API verifies token and activates account

This step ensures only verified users can log in and receive JWTs.

#### 5. Protecting APIs with JWT

Use JWT for authentication on protected views.

views.py:

```

-----
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response

@api_view(['GET'])
@permission_classes([IsAuthenticated])
def protected_view(request):
    return Response({'message': 'This is a protected API endpoint'})

```

#### 6. Refreshing Tokens

Access tokens expire after a short time for security.

Use the refresh endpoint to get a new access token:

```

POST /api/token/refresh/
{
  "refresh": "your_refresh_token"
}

```

## 7. Summary

- Use SimpleJWT for authentication in Django REST Framework
- Implement user registration and optional email verification
- Protect API endpoints with JWTAuthentication
- Use refresh tokens to extend user sessions securely